Proyecto Final

Definición, implementación e instanciación de clases

Anibal Yael Gonzalez Garcia

Codigo:220814268

Programación Orientada a Objetos – Ingenieria Informatica – Alfredo Gutierrez Hernandez

Codigo de materia: I5289

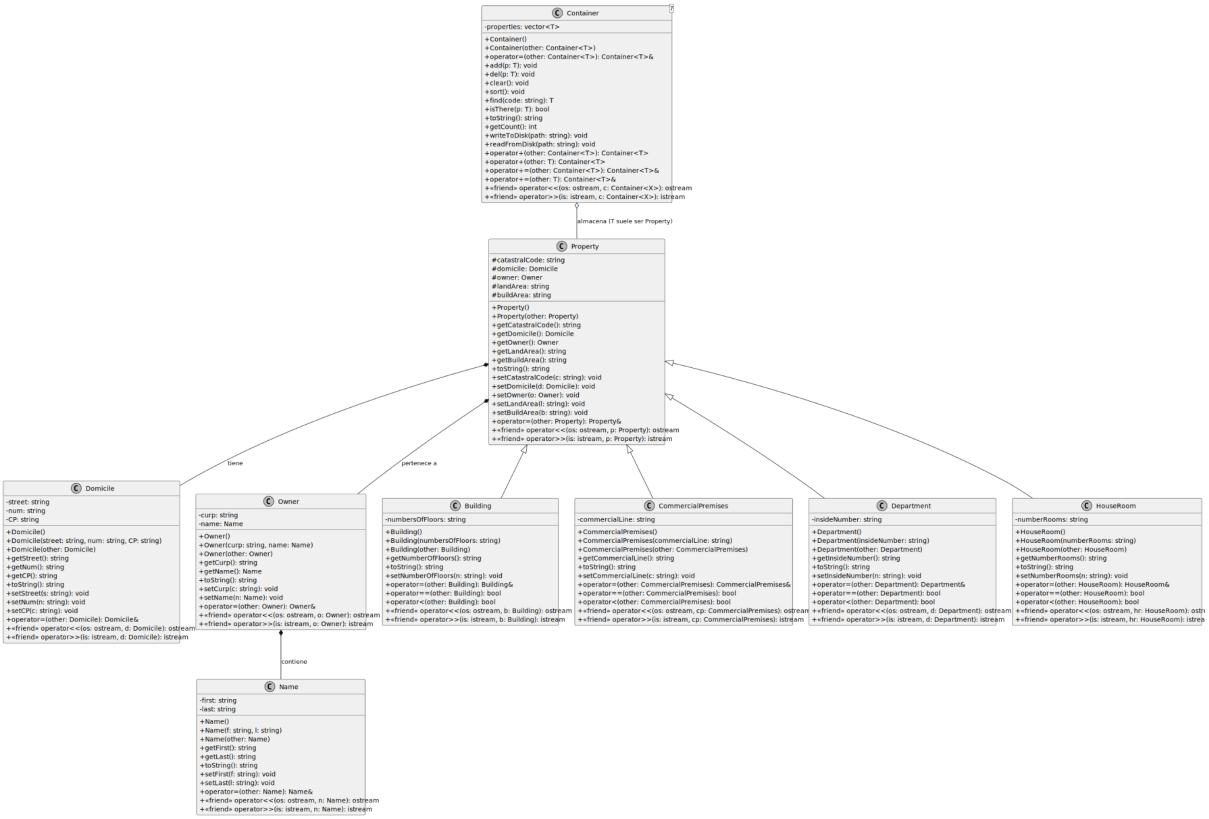
5 Mayo del 2025

Introducción

El presente proyecto consiste en el diseño y desarrollo de un sistema informático para la administración de bienes raíces utilizando el lenguaje C++. A través de la implementación de conceptos avanzados de POO como la herencia, el polimorfismo y la composición, se busca estructurar de manera lógica la información de diversos tipos de inmuebles (Casas, Locales, Edificios y Departamentos), garantizando un manejo técnico profesional de los datos catastrales y de propiedad.

Arquitectura y Diagrama de Clases

El siguiente diagrama muestra la jerarquia completa de clases del sistema, sus atributos, metodos y las relaciones de herencia, composicion y asociacion entre ellas.

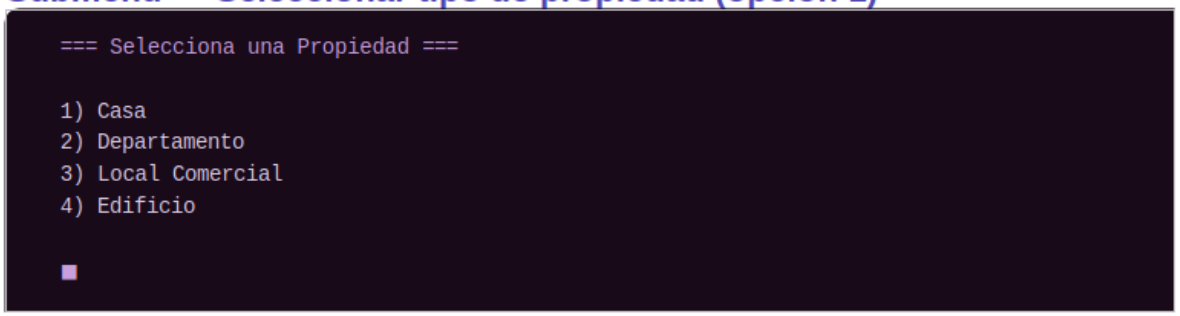


Pantallas del Programa

El programa corre en una terminal de Ubuntu con fondo oscuro. A continuacion se muestran las pantallas principales tal como aparecen durante la ejecucion.

Menu Principal

Submenu — Seleccionar tipo de propiedad (opcion 1)



Formulario de captura — Casa

```
=== Casa ===

Clave Catastral:
14-039-001-001-0001
=== Direccion ===
Calle:
Av. Juarez
Numero:
123
C.P:
44100
=== Dueno ===
Curp:
GOME910101HJCRRS
Nombre:
Andres
Apellido:
Gomez
Superficie de terreno:
120
Superficie Construida:
90
Numero Cuartos:
4

Propiedad agregada correctamente...
[Enter] para continuar...
```

Eliminar propiedad — con y sin resultado

Propiedad encontrada

```
Clave catastral:
14-039-001-001-0001
Estas seguro de eliminar S/N?
S

Propiedad eliminada...
[Enter] para continuar...
```

No encontrada

```
Clave Catastral:
99-999-999-999-9999

Propiedad no encontrada
[Enter] para continuar...
```

Guardar y Ordenar propiedades

Guardar (opcion 7)

```
Escribiendo al disco...
Propiedades guardadas
correctamente
```

Codigo Fuente

Building.hpp

```
#ifndef __BUILDING_H__
#define __BUILDING_H__

#include "Property.hpp"

class Building : public Property
{
private:
std::string numbersOfFloors;

public:
Building(/* args */);
Building(std::string numbersOfFloors);
Building(const Building &);

std::string getNumberOfFloors() const;

std::string toString() const;

void setNumberOfFloors(const std::string &);

Building &operator=(const Building &);

bool operator==(const Building &) const;
bool operator<(const Building &) const;

friend std::ostream &operator<<(std::ostream &, const Building &);
friend std::istream &operator>>(std::istream &, Building &);
};

#endif // __BUILDING_H__
```

Building.cpp

```cpp
#include "Building.hpp"

Building::Building(/* args */) : numbersOfFloors("")
{
}

Building::Building(std::string n) : numbersOfFloors(n)
{
}

Building::Building(const Building &other) : Property(other),
numbersOfFloors(other.numbersOfFloors)
{
}

std::string Building::getNumberOfFloors() const
{
return this->numbersOfFloors;
}

std::string Building::toString() const
{
return Property::toString().substr(0, Property::toString().length() - 1) + "
EDIF| " + numbersOfFloors + "\n";
}

void Building::setNumberOfFloors(const std::string &v)
{
this->numbersOfFloors = v;
}

Building &Building::operator=(const Building &other)
{
if (this != &other)
{
Property::operator=(other);
this->numbersOfFloors = other.numbersOfFloors;
}

return *this;
}

bool Building::operator==(const Building &other) const
{
return this->catastralCode == other.catastralCode;
}

bool Building::operator<(const Building &other) const
{
return this->catastralCode < other.catastralCode;
}

std::ostream &operator<<(std::ostream &os, const Building &b)
{
os << static_cast<const Property &>(b)
<< b.numbersOfFloors << '*';

return os;
}

std::istream &operator>>(std::istream &is, Building &b)
{
std::string tmp;

is >> static_cast<Property &>(b);
getline(is, b.numbersOfFloors, '*');

return is;}
```

CommercialPremises.hpp

```
  #ifndef __COMMERCIALPREMISES_H__
#define __COMMERCIALPREMISES_H__

#include "Property.hpp"
#include <string>
#include <iostream>
#include <fstream>
#include <sstream>

class CommercialPremises : public Property
{
private:
std::string commercialLine;

public:
CommercialPremises(/* args */);
CommercialPremises(std::string commercialLine);
CommercialPremises(const CommercialPremises &);

std::string getCommercialLine() const;

std::string toString() const;

void setCommercialLine(const std::string &);

CommercialPremises &operator=(const CommercialPremises &);

bool operator==(const CommercialPremises &) const;
bool operator<(const CommercialPremises &) const;

friend std::ostream &operator<<(std::ostream &, const CommercialPremises &);
friend std::istream &operator>>(std::istream &, CommercialPremises &);
};

#endif // __COMMERCIALPREMISES_H__
```

CommercialPremises.cpp

```
#include "CommercialPremises.hpp"

CommercialPremises::CommercialPremises(/* args */): commercialLine(""){}

CommercialPremises::CommercialPremises(std::string cL): commercialLine(cL){}

CommercialPremises::CommercialPremises(const CommercialPremises& other):
Property(other), commercialLine(other.commercialLine){}

std::string CommercialPremises::getCommercialLine() const
{
return this -> commercialLine;
}

std::string CommercialPremises::toString() const
{
return Property::toString().substr(0, Property::toString().length()-1) + "
LOCAL|" + commercialLine() + "$\n";
}

void CommercialPremises::setCommercialLine(const std::string & v)
{
this -> commercialLine = v;
}

CommercialPremises& CommercialPremises::operator = (const
CommercialPremises& other)
{
if(this != &other){
Property::operator=(other);
this -> commercialLine = other.commercialLine;
}

return *this;
}

bool CommercialPremises::operator == (const CommercialPremises& other) const
{
return this -> catastralCode == other.catastralCode;
}

bool CommercialPremises::operator < (const CommercialPremises& other) const
{
return this -> catastralCode < other.catastralCode;
}

std::istream& operator >> (std::istream&is, CommercialPremises &c)
{
std::string tmp;
is >> static_cast<Property&>(c);
getline(is, c.commercialLine,'*');

return is;
}

std::ostream& operator << (std::ostream &os, const CommercialPremises &c)
{
os << static_cast<const Property&>(c)
<< c.commercialLine << '*';

return os;
}
```

Department.hpp

```
#ifndef __DEPARTMENT_H__
#define __DEPARTMENT_H__

#include "Property.hpp"

class Department : public Property
{
private:
std::string insideNumber;
/* data */
public:
Department(/* args */);
Department(std::string insideNumber);
Department(const Department &);

std::string getInsideNumber() const;

std::string toString() const;

void setInsideNumber(const std::string &);

Department &operator=(const Department &);

bool operator==(const Department &) const;
bool operator<(const Department &) const;

friend std::ostream &operator<<(std::ostream &, const Department &);
friend std::istream &operator>>(std::istream &, Department &);
};

#endif // __DEPARTMENT_H__
```

Domicile.hpp

```
#ifndef __DOMICILIO_H__
#define __DOMICILIO_H__

#include <string>
#include <iostream>
#include <fstream>
#include <sstream>

class Domicile
{
private:
std::string street;
std::string num;
std::string CP;
/* data */
public:
Domicile(/* args */);
Domicile(std::string street, std::string num, std::string CP);
Domicile(const Domicile &);

std::string getStreet() const;
std::string getNum() const;
std::string getCP() const;

std::string toString() const;

void setStreet(const std::string &);
void setNum(const std::string &);
void setCP(const std::string &);

Domicile &operator=(const Domicile &);

friend std::ostream &operator<<(std::ostream &, const Domicile &);
friend std::istream &operator>>(std::istream &, Domicile &);
};

#endif // __DOMICILIO_H__
```

HouseRoom.hpp

```
#ifndef __HOUSEROOM_H__
#define __HOUSEROOM_H__

#include "Property.hpp"

class HouseRoom : public Property
{
private:
std::string numberRooms;

public:
HouseRoom();
HouseRoom(std::string numberRooms);
HouseRoom(const HouseRoom &);

std::string getNumberRooms() const;

std::string toString() const;

void setNumberRooms(const std::string &);

HouseRoom &operator=(const HouseRoom &);

bool operator==(const HouseRoom &) const;
bool operator<(const HouseRoom &) const;

friend std::ostream &operator<<(std::ostream &, const HouseRoom &);
friend std::istream &operator>>(std::istream &, HouseRoom &);
};

#endif // __HOUSEROOM_H__
```

Name.hpp

```
#ifndef __NAME_H__
#define __NAME_H__

#include <string>
#include <iostream>
#include <fstream>
#include <sstream>

class Name
{
private:
std::string first;
std::string last;
/* data */
public:
Name(/* args */);
Name(std::string f, std::string l);
Name(const Name &);

std::string getFirst() const;
std::string getLast() const;

std::string toString() const;

void setFirst(const std::string &);
void setLast(const std::string &);

Name &operator=(const Name &);

friend std::ostream &operator<<(std::ostream &, const Name &);
friend std::istream &operator>>(std::istream &, Name &);
};

#endif // __NAME_H__
```

Owner.hpp

```
#ifndef __OWNER_H__
#define __OWNER_H__

#include <string>
#include "Name.hpp"
#include "Domicile.hpp"

#include <iostream>
#include <fstream>
#include <sstream>

class Owner
{
private:
std::string curp;
Name name;

/* data */
public:
Owner(/* args */);
Owner(std::string curp, Name name);
Owner(const Owner &);

std::string getCurp() const;
Name getName() const;

std::string toString() const;

void setCurp(const std::string &);
void setName(const Name &);

Owner &operator=(const Owner &);

friend std::ostream &operator<<(std::ostream &, const Owner &);
friend std::istream &operator>>(std::istream &, Owner &);
};

#endif // __OWNER_H__
```

Property.hpp

```
#ifndef __PROPERTY_H__
#define __PROPERTY_H__

#include <string>
#include "Domicile.hpp"
#include "Owner.hpp"

class Property
{
protected:
std::string catastralCode;
Domicile domicile;
Owner owner;
std::string landArea;
std::string buildArea;

public:
Property();
Property(const Property &);

std::string getCatastralCode() const;
Domicile getDomicile() const;
Owner getOwner() const;
std::string getLandArea() const;
std::string getBuildArea() const;

std::string toString() const;

void setCatastralCode(const std::string &);
void setDomicile(const Domicile &);
void setOwner(const Owner &);
void setLandArea(const std::string &);
void setBuildArea(const std::string &);

Property &operator=(const Property &);

friend std::ostream &operator<<(std::ostream &, const Property &);
friend std::istream &operator>>(std::istream &, Property &);
};

#endif // __PROPERTY_H__
```

UI.hpp

```
#ifndef __UI_H__
#define __UI_H__

#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include "Building.hpp"
#include "Container.hpp"
#include "CommercialPremises.hpp"
#include "Department.hpp"
#include "HouseRoom.hpp"

typedef Container<Building> Build;
typedef Container<CommercialPremises> Commercial;
typedef Container<Department> Depa;
typedef Container<HouseRoom> House;

class Ui
{
private:
Build *buildRef;
Commercial *CommercialRef;
Depa *depaRef;
House *houseRef;

void mainUI();

// Menus de propiedades

void menuBuilding();

void menuCommercial();

void menuDepartment();

void menuHouse();

// menus

void addProperties();

void showProperties();

void delProperti();

void findProperti();

void sortProperties();

void delAll();

void writeToFile();

void readFromFile();

void enterToContinue();

public:
Ui(Build &, Commercial &, Depa &, House &);
};

#endif // __UI_H__
```

Building.cpp

```
#include "Building.hpp"

Building::Building(/* args */) : numbersOfFloors("")
{
}

Building::Building(std::string n) : numbersOfFloors(n)
{
}

Building::Building(const Building &other) : Property(other),
numbersOfFloors(other.numbersOfFloors)
{
}

std::string Building::getNumberOfFloors() const
{
return this->numbersOfFloors;
}

std::string Building::toString() const
{
return Property::toString() + "Number of floors: " + this->numbersOfFloors + "\n";
}

void Building::setNumberOfFloors(const std::string &v)
{
this->numbersOfFloors = v;
}

Building &Building::operator=(const Building &other)
{
if (this != &other)
{
Property::operator=(other);
this->numbersOfFloors = other.numbersOfFloors;
}

return *this;
}

bool Building::operator==(const Building &other) const
{
return this->catastralCode == other.catastralCode;
}

bool Building::operator<(const Building &other) const
{
return this->catastralCode < other.catastralCode;
}

std::ostream &operator<<(std::ostream &os, const Building &b)
{
os << static_cast<const Property &>(b)
<< b.numbersOfFloors << '*';

return os;
}

std::istream &operator>>(std::istream &is, Building &b)
{
std::string tmp;

is >> static_cast<Property &>(b);
getline(is, b.numbersOfFloors, '*');

return is;
}
```

CommercialPremises.cpp

```cpp
#include "CommercialPremises.hpp"

CommercialPremises::CommercialPremises(/* args */) : commercialLine("")
{
}

CommercialPremises::CommercialPremises(std::string cL) : commercialLine(cL)
{
}

CommercialPremises::CommercialPremises(const CommercialPremises &other) :
{
}

std::string CommercialPremises::getCommercialLine() const
{
return this->commercialLine;
}

std::string CommercialPremises::toString() const
{
return "Commercial Line: " + commercialLine + "\n" + Property::toString().substr(0, Property::toString().length() - 1) + "
}

void CommercialPremises::setCommercialLine(const std::string &v)
{
this->commercialLine = v;
}

CommercialPremises &CommercialPremises::operator=(const CommercialPremises &other)
{
if (this != &other)
{
Property::operator=(other);
this->commercialLine = other.commercialLine;
}

return *this;
}

bool CommercialPremises::operator==(const CommercialPremises &other) const
{
return this->catastralCode == other.catastralCode;
}

bool CommercialPremises::operator<(const CommercialPremises &other) const
{
return this->catastralCode < other.catastralCode;
}

std::istream &operator>>(std::istream &is, CommercialPremises &c)
{
std::string tmp;
is >> static_cast<Property &>(c);
getline(is, c.commercialLine, '*');

return is;
}

std::ostream &operator<<(std::ostream &os, const CommercialPremises &c)
{

os << static_cast<const Property &>(c)
<< c.commercialLine << '*';

return os;
}
```

Department.cpp

```
#include "Department.hpp"

Department::Department(/* args */) : insideNumber("")
{
}

Department::Department(std::string i) : insideNumber(i)
{
}

Department::Department(const Department &other) : Property(other),
insideNumber(other.insideNumber)
{
}

std::string Department::getInsideNumber() const
{
return this->insideNumber;
}

std::string Department::toString() const
{
return Property::toString().substr(0, Property::toString().length() - 1) + "
, Inside number: " + insideNumber + "}";
}

void Department::setInsideNumber(const std::string &v)
{
this->insideNumber = v;
}

Department &Department::operator=(const Department &other)
{
if (this != &other)
{
Property::operator=(other);
this->insideNumber = other.insideNumber;
}
return *this;
}

bool Department::operator==(const Department &other) const
{
return this->catastralCode == other.catastralCode;
}

bool Department::operator<(const Department &other) const
{
return this->catastralCode < other.catastralCode;
}

std::istream &operator>>(std::istream &is, Department &d)
{
std::string tmp;

is >> static_cast<Property &>(d);
getline(is, d.insideNumber, '*');

return is;
}

std::ostream &operator<<(std::ostream &os, const Department &d)
{
os << static_cast<const Property &>(d)
<< d.insideNumber << '*';

return os;
}
```

Domicile.cpp

```
#include "Domicile.hpp"

using namespace std;

Domicile::Domicile(/* args */) : street(""), num(""), CP("")
{
}

Domicile::Domicile(std::string c, std::string n, std::string p) : street(c),
num(n), CP(p)
{
}

Domicile::Domicile(const Domicile &other) : street(other.street),
num(other.num), CP(other.CP)
{
}

std::string Domicile::getStreet() const
{
return this->street;
}

std::string Domicile::getNum() const
{
return this->num;
}

std::string Domicile::getCP() const
{
return this->CP;
}

std::string Domicile::toString() const
{
return street + ", " + num + ", CP: " + CP;
}

void Domicile::setStreet(const std::string &v)
{
this->street = v;
}

void Domicile::setNum(const std::string &v)
{
this->num = v;
}

void Domicile::setCP(const std::string &v)
{
this->CP = v;
}

Domicile &Domicile::operator=(const Domicile &other)
{
if (this != &other)
{
this->street = other.street;
this->num = other.num;
this->CP = other.CP;
}

return *this;
}

std::istream &operator>>(std::istream &is, Domicile &d)
{
string tmp;

getline(is, d.street, '*');
```

```
getline(is, d.num, '*');
getline(is, d.CP, '*');

return is;
}

std::ostream &operator<<(std::ostream &os, const Domicile &d)
{
os << d.street << '*'
<< d.num << '*'
<< d.CP;

return os;
}
```

HouseRoom.cpp

```
#include "HouseRoom.hpp"

HouseRoom::HouseRoom(/* args */) : numberRooms("")
{
}

HouseRoom::HouseRoom(std::string n) : numberRooms(n)
{
}

HouseRoom::HouseRoom(const HouseRoom &other) : Property(other),
numberRooms(other.numberRooms)
{
}

std::string HouseRoom::getNumberRooms() const
{
return this->numberRooms;
}

std::string HouseRoom::toString() const
{
return Property::toString().substr(0, Property::toString().length() - 1) + "
CASA | " + numberRooms + " hab\n";
}

void HouseRoom::setNumberRooms(const std::string &v)
{
this->numberRooms = v;
}

HouseRoom &HouseRoom::operator=(const HouseRoom &other)
{
if (this != &other)
{
Property::operator=(other);
this->numberRooms = other.numberRooms;
}

return *this;
}

bool HouseRoom::operator==(const HouseRoom &other) const
{
return this->catastralCode == other.catastralCode;
}

bool HouseRoom::operator<(const HouseRoom &other) const
{
```

```
return this->catastralCode < other.catastralCode;
}

std::istream &operator>>(std::istream &is, HouseRoom &h)
{
std::string tmp;
is >> static_cast<Property &>(h);
getline(is, h.numberRooms, '*');

return is;
}

std::ostream &operator<<(std::ostream &os, const HouseRoom &h)
{
os << static_cast<const Property &>(h)
<< h.numberRooms << '*';

return os;
}
```

Name.cpp

```
#include "Name.hpp"

Name::Name(/* args */) : first(""), last("")
{
}

Name::Name(std::string f, std::string l) : first(f), last(l)
{
}

Name::Name(const Name &other) : first(other.first), last(other.last)
{
}

std::string Name::getFirst() const
{
return this->first;
}

std::string Name::getLast() const
{
return this->last;
}

std::string Name::toString() const
{
return first + " " + last;
}

void Name::setFirst(const std::string &v)
{
this->first = v;
}

void Name::setLast(const std::string &v)
{
this->last = v;
}

Name &Name::operator=(const Name &other)
{

if (this != &other)
```

```
{
this->first = other.first;
this->last = other.last;
}

return *this;
}

std::istream &operator>>(std::istream &is, Name &n)
{
getline(is, n.first, '*');
getline(is, n.last, '*');

return is;
}

std::ostream &operator<<(std::ostream &os, const Name &n)
{
os << n.first << '*'
<< n.last;

return os;
}
```

Owner.cpp

```
#include "Owner.hpp"

Owner::Owner(/* args */) : curp("")
{
}

Owner::Owner(std::string c, Name n) : curp(c), name(n)
{
}

Owner::Owner(const Owner &other) : curp(other.curp), name(other.name)
{
}

std::string Owner::getCurp() const
{
return this->curp;
}

Name Owner::getName() const
{
return this->name;
}

std::string Owner::toString() const
{
return "[" + curp + "] " + name.toString();
}

void Owner::setCurp(const std::string &v)
{
this->curp = v;
}

void Owner::setName(const Name &v)
{
this->name = v;
}
```

```
Owner &Owner::operator=(const Owner &other)
{
if (this != &other)
{
this->curp = other.curp;
this->name = other.name;
}
return *this;
}

std::istream &operator>>(std::istream &is, Owner &o)
{
std::string tmp;

getline(is, o.curp, '*');
is >> o.name;

return is;
}

std::ostream &operator<<(std::ostream &os, const Owner &o)
{
os << o.curp << '*'
<< o.name;

return os;
}
```

Property.cpp

```
#include "Property.hpp"
#include <iostream>
#include <fstream>

Property::Property()
{
}

Property::Property(const Property &other) :
catastralCode(other.catastralCode), domicile(other.domicile), owner(
other.owner), landArea(other.landArea), buildArea(other.buildArea)
{
}

std::string Property::getCatastralCode() const
{
return this->catastralCode;
}

Domicile Property::getDomicile() const
{
return this->domicile;
}

Owner Property::getOwner() const
{
return this->owner;
}
```

```
std::string Property::getLandArea() const
{
return this->landArea;
}

std::string Property::getBuildArea() const
{
return this->buildArea;
}

std::string Property::toString() const
{
return "| " + catastralCode + " | " + domicile.toString() + " | " +
owner.toString() + " | " + landArea + "m2 | " + buildArea + "m2 |\n";
}

void Property::setCatastralCode(const std::string &v)
{
this->catastralCode = v;
}

void Property::setDomicile(const Domicile &v)
{
this->domicile = v;
}

void Property::setOwner(const Owner &v)
{
this->owner = v;
}

void Property::setLandArea(const std::string &v)
{
this->landArea = v;
}

void Property::setBuildArea(const std::string &v)
{
this->buildArea = v;
}

Property &Property::operator=(const Property &other)
{
if (this != &other)
{
this->catastralCode = other.catastralCode;
this->domicile = other.domicile;
this->owner = other.owner;
this->landArea = other.landArea;
this->buildArea = other.buildArea;
}
return *this;
}

std::istream &operator>>(std::istream &is, Property &p)
{
getline(is, p.catastralCode, '*');
is >> p.domicile;
is >> p.owner;
getline(is, p.landArea, '*');
getline(is, p.buildArea, '*');

return is;
}

std::ostream &operator<<(std::ostream &os, const Property &p)
{
os << p.catastralCode << '*'
<< p.domicile << '*'
```

```
<< p.owner << '*'
<< p.landArea << '*'
<< p.buildArea << '*';

return os;
}
```

UI.cpp

```
  #include "Ui.hpp"
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include "Container.hpp"

using namespace std;

void Ui::mainUI()
{
int op = 0;

do
{

cout << "=== Menu Principal ===" << endl
<< endl;

cout << "1) Anadir Propiedad" << endl;
cout << "2) Eliminar Propiedad" << endl;
cout << "3) Buscar Propiedad" << endl;
cout << "4) Ordenar Propiedades" << endl;
cout << "5) Limpiar Propiedades" << endl;
cout << "6) Mostrar Propiedades" << endl;
cout << "7) Guardar Propiedades" << endl;
cout << "8) Salir" << endl
<< endl;

cin >> op;

switch (op)
{
case 1:
addProperties();
break;

case 2:
delProperti();
break;

case 3:
findProperti();
break;

case 4:
sortProperties();
break;

case 5:
delAll();
break;

case 6:
showProperties();
break;
```

```
case 7:
writeToFile();
break;

case 8:
break;

default:

break;
}

} while (op != 8);
}

void Ui::addProperties()
{
system("clear");
int op = 0;

cout << "=== Selecciona una Propiedad ===" << endl
<< endl;

cout << "1) Casa" << endl;
cout << "2) Departamento" << endl;
cout << "3) Local Comercial" << endl;
cout << "4) Edificio" << endl;

cin >> op;

switch (op)
{
case 1:
menuHouse();
break;
case 2:
menuDepartment();
break;
case 3:
menuCommercial();
break;
case 4:
menuBuilding();
break;
default:
break;
}
}

/// Menus de propiedades

void Ui::menuBuilding()
{
system("clear");
Building building;
Domicile domicile;
Owner owner;
Name name;

string cadena;

cout << "=== Edificio ===" << endl
<< endl;
cout << "Clave Catastral: " << endl;
getline(cin >> ws, cadena);
building.setCatastralCode(cadena);

cout << "=== Direccion ===" << endl;
```

```
cout << "Calle : " << endl;
getline(cin >> ws, cadena);
domicile.setStreet(cadena);

cout << "Numero: " << endl;
getline(cin >> ws, cadena);
domicile.setNum(cadena);

cout << "C.P: " << endl;
getline(cin >> ws, cadena);
domicile.setCP(cadena);

building.setDomicile(domicile);

cout << "=== Dueno ===" << endl;
cout << "Curp: " << endl;
getline(cin >> ws, cadena);
owner.setCurp(cadena);

cout << "Nombre: " << endl;
getline(cin >> ws, cadena);
name.setFirst(cadena);

cout << "Apellido: " << endl;
getline(cin >> ws, cadena);
name.setLast(cadena);

owner.setName(name);

building.setOwner(owner);

cout << "Superficie de terreno: " << endl;
getline(cin >> ws, cadena);
building.setLandArea(cadena);

cout << "Supericie Construida: " << endl;
getline(cin >> ws, cadena);
building.setBuildArea(cadena);

cout << "Numero de Pisos: " << endl;
getline(cin >> ws, cadena);
building.setNumberOfFloors(cadena);

buildRef->add(building);

cout << "Propiedad agregada correctamente...\n";
enterToContinue();
}

void Ui::menuCommercial()
{
system("clear");
CommercialPremises store;
Domicile domicile;
Owner owner;
Name name;

string cadena;

cout << "=== Local ===" << endl
<< endl;
cout << "Clave Catastral: " << endl;
getline(cin >> ws, cadena);
store.setCatastralCode(cadena);

cout << "=== Direccion ===" << endl;
cout << "Calle : " << endl;
getline(cin >> ws, cadena);
domicile.setStreet(cadena);
```

```
cout << "Numero: " << endl;
getline(cin >> ws, cadena);
domicile.setNum(cadena);

cout << "C.P: " << endl;
getline(cin >> ws, cadena);
domicile.setCP(cadena);

store.setDomicile(domicile);

cout << "=== Dueno ===" << endl;
cout << "Curp: " << endl;
getline(cin >> ws, cadena);
owner.setCurp(cadena);

cout << "Nombre: " << endl;
getline(cin >> ws, cadena);
name.setFirst(cadena);

cout << "Apellido: " << endl;
getline(cin >> ws, cadena);
name.setLast(cadena);

owner.setName(name);

store.setOwner(owner);

cout << "Superficie de terreno: " << endl;
getline(cin >> ws, cadena);
store.setLandArea(cadena);

cout << "Supericie Construida: " << endl;
getline(cin >> ws, cadena);
store.setBuildArea(cadena);

cout << "Giro Comercial: " << endl;
getline(cin >> ws, cadena);
store.setCommercialLine(cadena);

CommercialRef->add(store);

cout << "Propiedad agregada correctamente...\n";
enterToContinue();
}

void Ui::menuDepartment()
{
system("clear");
Department department;
Domicile domicile;
Owner owner;
Name name;

string cadena;

cout << "=== Departamento ===" << endl
<< endl;
cout << "Clave Catastral: " << endl;
getline(cin >> ws, cadena);
department.setCatastralCode(cadena);

cout << "=== Direccion ===" << endl;
cout << "Calle : " << endl;
getline(cin >> ws, cadena);
domicile.setStreet(cadena);

cout << "Numero: " << endl;
```

```
getline(cin >> ws, cadena);
domicile.setNum(cadena);

cout << "C.P: " << endl;
getline(cin >> ws, cadena);
domicile.setCP(cadena);

department.setDomicile(domicile);

cout << "=== Dueno ===" << endl;
cout << "Curp: " << endl;
getline(cin >> ws, cadena);
owner.setCurp(cadena);

cout << "Nombre: " << endl;
getline(cin >> ws, cadena);
name.setFirst(cadena);

cout << "Apellido: " << endl;
getline(cin >> ws, cadena);
name.setLast(cadena);

owner.setName(name);

department.setOwner(owner);

cout << "Superficie de terreno: " << endl;
getline(cin >> ws, cadena);
department.setLandArea(cadena);

cout << "Supericie Construida: " << endl;
getline(cin >> ws, cadena);
department.setBuildArea(cadena);

cout << "Numero Interior: " << endl;
getline(cin >> ws, cadena);
department.setInsideNumber(cadena);

depaRef->add(department);

cout << "Propiedad agregada correctamente...\n";
enterToContinue();
}

void Ui::menuHouse()
{
system("clear");
HouseRoom house;
Domicile domicile;
Owner owner;
Name name;

string cadena;

cout << "=== Casa ===" << endl
<< endl;
cout << "Clave Catastral: " << endl;
getline(cin >> ws, cadena);
house.setCatastralCode(cadena);

cout << "=== Direccion ===" << endl;
cout << "Calle : " << endl;
getline(cin >> ws, cadena);
domicile.setStreet(cadena);

cout << "Numero: " << endl;
getline(cin >> ws, cadena);
domicile.setNum(cadena);
```

```
cout << "C.P: " << endl;
getline(cin >> ws, cadena);
domicile.setCP(cadena);

house.setDomicile(domicile);

cout << "=== Dueno ===" << endl;
cout << "Curp: " << endl;
getline(cin >> ws, cadena);
owner.setCurp(cadena);

cout << "Nombre: " << endl;
getline(cin >> ws, cadena);
name.setFirst(cadena);

cout << "Apellido: " << endl;
getline(cin >> ws, cadena);
name.setLast(cadena);

owner.setName(name);

house.setOwner(owner);

cout << "Superficie de terreno: " << endl;
getline(cin >> ws, cadena);
house.setLandArea(cadena);

cout << "Supericie Construida: " << endl;
getline(cin >> ws, cadena);
house.setBuildArea(cadena);

cout << "Numero Cuartos: " << endl;
getline(cin >> ws, cadena);
house.setNumberRooms(cadena);

houseRef->add(house);

cout << "Propiedad agregada correctamente...\n";

enterToContinue();
}

void Ui::showProperties()
{

system("clear");
cout << "Mostrando Propiedades..." << endl
<< endl;

bool anyDisplayed = false;

if (houseRef->getCount() > 0)
{
cout << "=== Casas ===" << endl
<< string(60, '-') << endl;
cout << houseRef->toString() << endl;
anyDisplayed = true;
}

if (depaRef->getCount() > 0)
{
cout << "=== Departamentos ===" << endl
<< string(60, '-') << endl;
cout << depaRef->toString() << endl;
anyDisplayed = true;
}
```

```
if (CommercialRef->getCount() > 0)
{
cout << "=== Locales Comerciales ===" << endl
<< string(60, '-') << endl;
cout << CommercialRef->toString() << endl;
anyDisplayed = true;
}

if (buildRef->getCount() > 0)
{
cout << "=== Edificios ===" << endl
<< string(60, '-') << endl;
cout << buildRef->toString() << endl;
anyDisplayed = true;
}

if (!anyDisplayed)
{
cout << "No hay propiedades registradas." << endl;
}

cout << "Propiedades mostradas correctamente..." << endl;
cin.get();
enterToContinue();
}

void Ui::delProperti()
{
system("clear");
string cadena;
string op;

CommercialPremises commerial;
Building building;
Department department;
HouseRoom house;

cout << "Clave catastral: " << endl;
getline(cin >> ws, cadena);

cout << "Estas seguro de eliminar la propiedad S/N?" << endl;
cin >> op;

if (op != "N")
{
commerial.setCatastralCode(cadena);
building.setCatastralCode(cadena);
department.setCatastralCode(cadena);
house.setCatastralCode(cadena);

CommercialRef->del(commerial);
buildRef->del(building);
depaRef->del(department);
houseRef->del(house);

cout << "Propiedad eliminada..." << endl;

enterToContinue();
}
}

void Ui::findProperti()
{
system("clear");
string cadena;
CommercialPremises commerial;
Building building;
Department department;
HouseRoom house;
```

```
cout << "Clave Catastral: " << endl;
getline(cin >> ws, cadena);
commerial.setCatastralCode(cadena);
building.setCatastralCode(cadena);
department.setCatastralCode(cadena);
house.setCatastralCode(cadena);

if (this->CommercialRef->isThere(commerial))
{
CommercialPremises found = CommercialRef->find(cadena);
cout << found.toString();
}
else if (this->buildRef->isThere(building))
{
Building found = buildRef->find(cadena);
cout << found.toString();
}
else if (this->depaRef->isThere(department))
{
Department found = depaRef->find(cadena);
cout << found.toString();
}
else if (this->houseRef->isThere(house))
{
HouseRoom found = houseRef->find(cadena);
cout << found.toString();
}
else
{
cout << "\nPropiedad no encontrada";
}

enterToContinue();
}

void Ui::sortProperties()
{
system("clear");

cout << "Ordenando..." << endl;
this->houseRef->sort();
this->depaRef->sort();
this->CommercialRef->sort();
this->buildRef->sort();

cout << "Registro ordenado " << endl;
enterToContinue();
}

void Ui::delAll()
{
system("clear");
string op;

cout << "Estas seguro de eliminar todos los registros S/N?" << endl;
cin >> op;

if (op != "N")
{
CommercialRef->clear();
buildRef->clear();
depaRef->clear();
houseRef->clear();

enterToContinue();
}
}
```

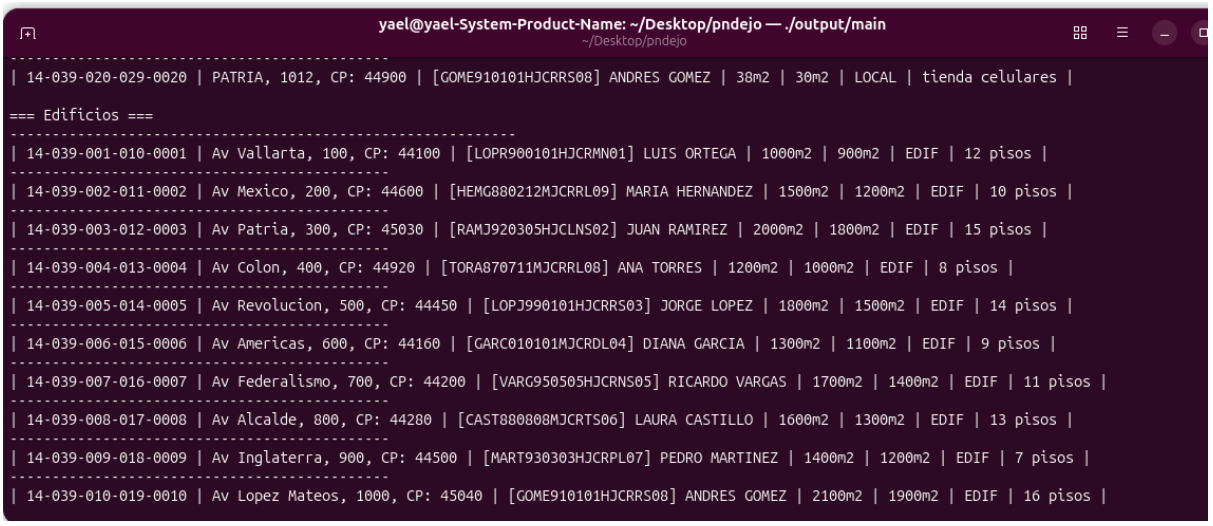
```
void Ui::writeToFile()
{
system("clear");
cout << "Escribiendo al disco..." << endl;

ofstream houseFIle("house.file", ios_base::trunc);
ofstream depaFile("depa.file", ios_base::trunc);
ofstream commercialFile("commercial.file", ios_base::trunc);
ofstream buildFIle("build.file", ios_base::trunc);

if (houseFIle.is_open())
{
houseFIle << *houseRef;
houseFIle.close();

if (depaFile.is_open())
{
depaFile << *depaRef;
depaFile.close();

if (commercialFile.is_open())
{
commercialFile << *CommercialRef;
commercialFile.close();

if (buildFIle.is_open())
{
buildFIle << *buildRef;
buildFIle.close();
}
}
}

cout << "Propiedades guardadas correctamente" << endl;
enterToContinue();
}
else
{
cout << "error" << endl;
}
}

void Ui::readFromFile()
{

ifstream house("house.file");
ifstream depa("depa.file");
ifstream commercial("commercial.file");
ifstream building("build.file");

if (house.is_open())
{
house >> *houseRef;
house.close();
}
if (depa.is_open())
{
depa >> *depaRef;
depa.close();
}
if (commercial.is_open())
{
commercial >> *CommercialRef;
commercial.close();
}
if (building.is_open())
{
building >> *buildRef;
building.close();
```

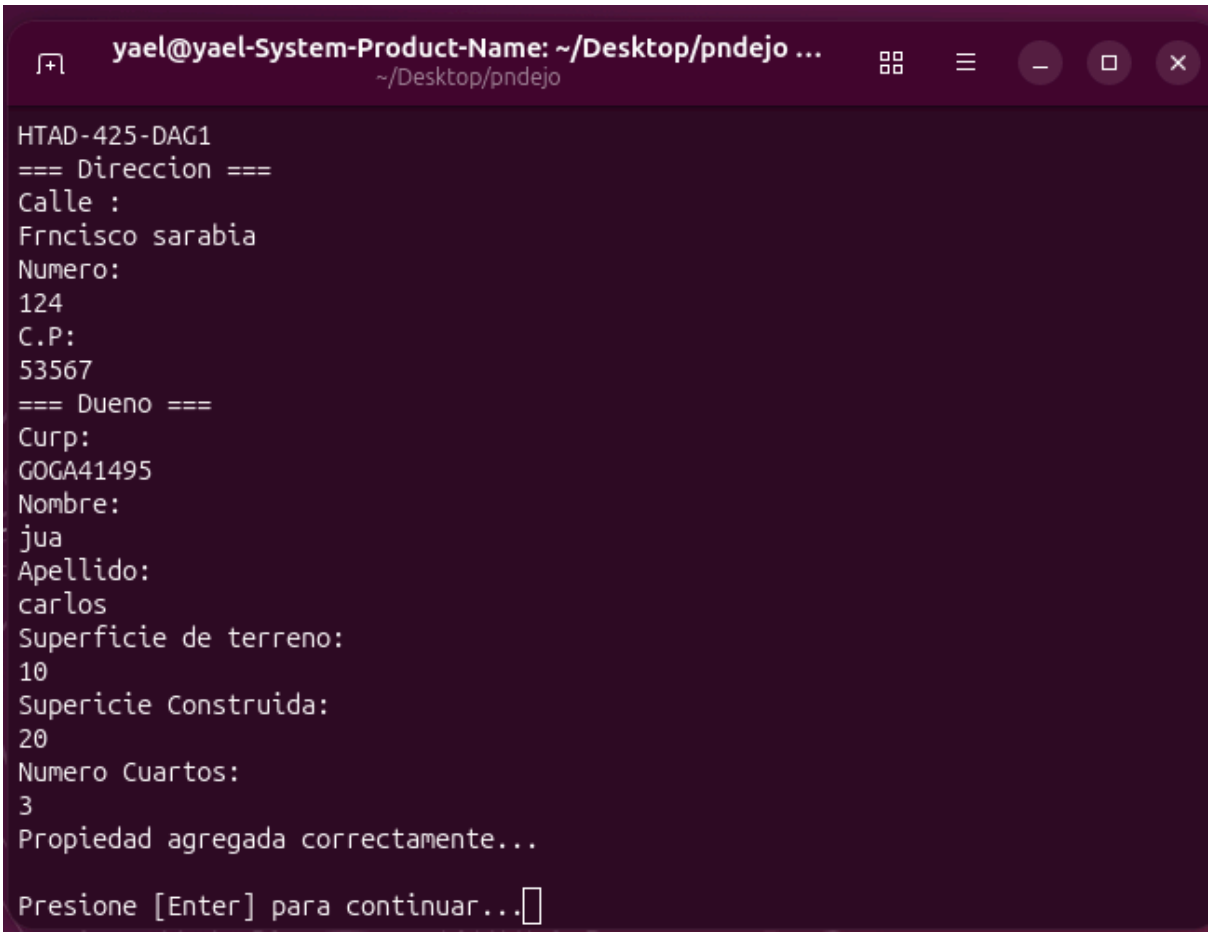
```
}
}

void Ui::enterToContinue()
{
cout << "\nPresione [Enter] para continuar...";
cin.get();
system("clear");
}

Ui::Ui(Build &b, Commercial &c, Depa &d, House &h)
: buildRef(&b), CommercialRef(&c), depaRef(&d), houseRef(&h)
{
readFromFile();
mainUI();
}
```

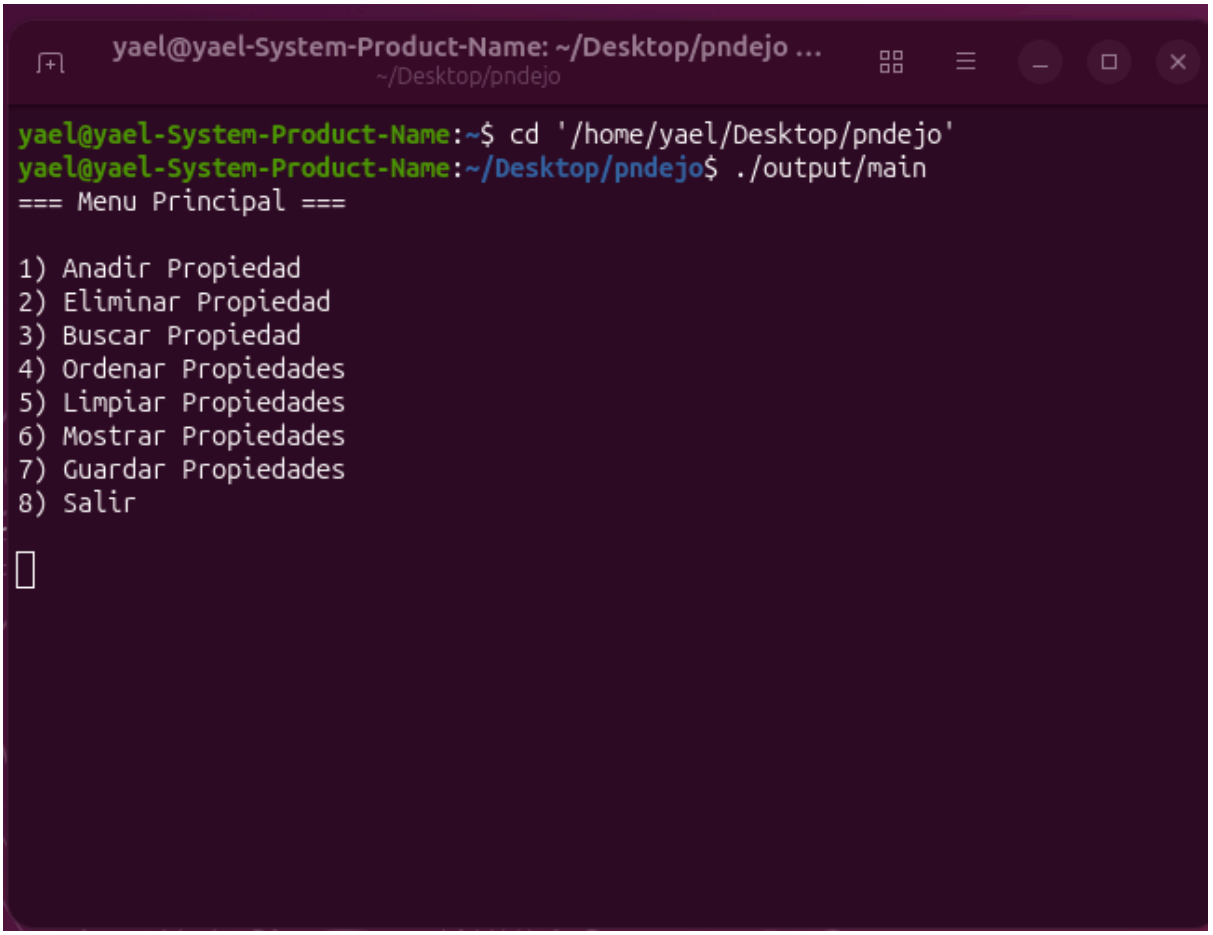
UI.cpp

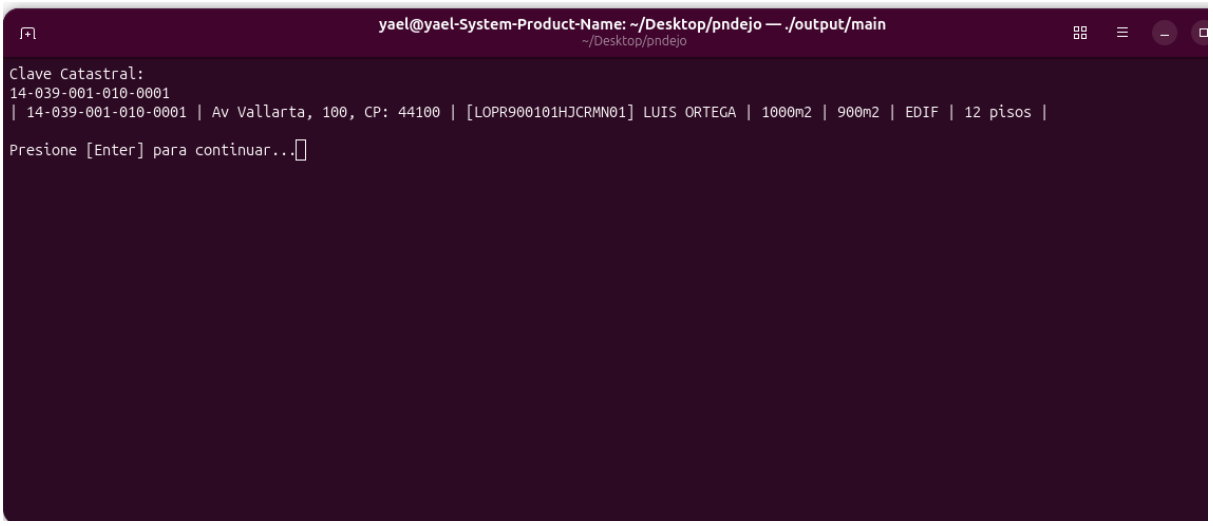
```
#include <iostream>

#include "Property.hpp"
#include "Building.hpp"
#include "CommercialPremises.hpp"
#include "Department.hpp"
#include "HouseRoom.hpp"
#include "Property.hpp"
#include "Container.hpp"
#include "Ui.hpp"

using namespace std;

int main()
{
Container<Building> myBuilding;
Container<CommercialPremises> myCommercial;
Container<HouseRoom> myHouse;
Container<Department> myDepartment;

Ui myInterface(myBuilding, myCommercial, myDepartment, myHouse);
return 0;
}
```

C apturas te pantalla






yael@yael-System-Product-Name: ~/Desktop/pndejo ...
~/Desktop/pndejo
yael@yael-System-Product-Name:~$ cd '/home/yael/Desktop/pndejo'
yael@yael-System-Product-Name:~/Desktop/pndejo$ ./output/main
=== Menu Principal ===
1) Anadir Propiedad
2) Eliminar Propiedad
3) Buscar Propiedad
4) Ordenar Propiedades
5) Limpiar Propiedades
6) Mostrar Propiedades
7) Guardar Propiedades
8) Salir


yael@yael-System-Product-Name: ~/Desktop/pndejo — ./output/main
~/Desktop/pndejo
Clave Catastral:
14-039-001-010-0001
| 14-039-001-010-0001 | Av Vallarta, 100, CP: 44100 | [LOPR900101HJCRMN01] LUIS ORTEGA | 1000m2 | 900m2 | EDIF | 12 pisos |
Presione [Enter] para continuar...

Conclusión Personal

La realización de este proyecto me ha permitido reafirmar que la Programación Orientada a Objetos no es solo una técnica de escritura, sino una metodología para organizar la complejidad del mundo real. Al implementar la herencia, logré reducir la redundancia de código, permitiendo que atributos compartidos como la clave catastral o la superficie se gestionen desde una base común, mientras que la composición facilitó el manejo de datos complejos como los domicilios y la información de los propietarios de forma modular.

Uno de los mayores retos fue la gestión de la clase contenedora. La capacidad de ordenar y filtrar elementos de distintos tipos bajo una misma estructura demuestra el poder del polimorfismo, simplificando tareas que de otro modo requerirían procesos separados para cada categoría de inmueble. Finalmente, la integración de la persistencia en disco transforma esta aplicación de un simple ejercicio académico en una herramienta funcional y persistente, capaz de mantener la integridad de la información a largo plazo.